

CHAPTER 1

1.1 Abstract

The Secure Messaging System using Hashing is a cybersecurity-based application developed to provide secure communication between users. The system ensures that messages are transmitted safely and any unauthorized modification can be detected. In traditional messaging systems, data security and integrity are not always guaranteed, which may lead to message tampering without user awareness.

This system allows users to register, login, and exchange messages securely. Each message is processed using a hashing algorithm (SHA-256) before being stored, which helps in maintaining data integrity. The system verifies the message at the receiver's end and identifies whether the message is secure or tampered.

The system is developed using Node.js, Express.js, MongoDB, HTML, CSS, and Bootstrap. MongoDB is used to store user details and message data efficiently.

This system helps in improving communication security, ensuring data integrity, and demonstrating the practical implementation of cybersecurity concepts in real-world applications.

1.2 Introduction To System

The Secure Messaging System using Hashing is a cybersecurity-based application developed to provide secure and reliable communication between users. In traditional messaging systems, messages are transmitted without strong mechanisms to ensure data integrity and security. This increases the risk of unauthorized access, data modification, and message tampering without the knowledge of users. It also becomes difficult to verify whether the received message is original or altered.

To overcome these issues, the Secure Messaging System provides a secure solution where users can register, login, and exchange messages safely. The system uses hashing techniques (SHA-256) to convert messages into a fixed hash value before storing them. This helps in maintaining data integrity and ensures that any change in the message can be detected easily. The system verifies the message at the receiver's end and displays whether the message is secure or tampered.

The system is developed using Node.js and Express.js for backend development, HTML, CSS, and Bootstrap for frontend design, and MongoDB for database management. It ensures secure data handling and efficient message processing. Overall, the Secure Messaging System improves communication security, reduces the risk of data tampering, and demonstrates the practical implementation of cybersecurity concepts in real-world applications.

1.3 Scope Of The System

The scope of the Secure Messaging System using Hashing includes the development of a cybersecurity-based application to provide secure communication between users. The system allows users to register, login, and exchange messages in a secure environment. It ensures that all messages are protected using hashing techniques, which helps in maintaining data integrity and preventing unauthorized modification.

The system maintains all user and message data in a centralized database, making it easy to store and retrieve information efficiently. It also verifies the integrity of messages at the receiver's end and indicates whether the message is secure or tampered. The system improves communication security by reducing the risk of data breaches and ensuring reliable message transmission.

The system is designed for users who require secure communication and can be applied in areas where data integrity is important. In the future, the system can be enhanced by adding features such as end-to-end encryption, file sharing, and mobile application support.

1.4 Objectives of the Study

The main objective of the Secure Messaging System using Hashing is to develop a secure communication system that ensures data integrity and protects messages from unauthorized modification. This system replaces insecure communication methods with a secure and reliable solution that is easy to use and efficient.

To Provide Secure Communication

One of the primary objectives of this project is to provide secure communication between users. In many traditional messaging systems, messages can be intercepted or modified. This system ensures that communication is safe and protected from such risks.

To Ensure Data Integrity Using Hashing

Another important objective is to maintain the integrity of messages using hashing techniques such as SHA-256. Each message is converted into a hash value before storage, ensuring that even a small change in the message can be detected easily.

To Detect Message Tampering

The system is designed to detect any unauthorized modification of messages. When a message is received, its hash is verified, and the system indicates whether the message is secure or tampered. This helps users trust the communication.

To Provide User Authentication

The system allows users to register and login securely. Authentication ensures that only authorized users can access the system and exchange messages, improving overall security.

To Store Data Securely

Another objective is to store user and message data securely in the database. The system ensures that all data is managed efficiently and can be retrieved when needed without compromising security.

To Provide a User-Friendly Interface

The system is designed with a simple and easy-to-use interface so that users can send and receive messages without difficulty. This improves usability and user experience.

To Demonstrate Cybersecurity Concepts

The project aims to demonstrate the practical implementation of cybersecurity concepts such as hashing, data integrity, and secure communication in real-world applications.

1.5 Purpose Of The System

The main purpose of the Secure Messaging System using Hashing is to provide a secure and reliable solution for communication between users. The system is designed to replace traditional messaging methods that do not guarantee data integrity and security, which may lead to unauthorized access and message tampering. By using this system, users can exchange messages in a safe and protected environment.

The system provides an easy interface for users to register, login, and send messages securely. It ensures that every message is processed using hashing techniques before storage, which helps in detecting any modification in the message. The system verifies the integrity of the message at the receiver's end and displays whether the message is secure or tampered. All the data is stored securely in the database, making it easy to manage and access information efficiently.

Overall, the purpose of the system is to improve communication security, ensure data integrity, reduce the risk of message tampering, and demonstrate the practical implementation of cybersecurity concepts in real-world applications.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 Software Requirements

Software requirements define the software tools and technologies required for the development and execution of the Secure Messaging System using Hashing. The system is developed using modern web technologies to ensure **efficiency, security, and reliability**.

The backend of the system is developed using **Node.js** with the Express.js framework, which helps in building fast and secure server-side applications. The frontend of the system is designed using **HTML, CSS, and Bootstrap**, which provide a simple, user-friendly interface and responsive design.

The system uses MongoDB database to store all the information such as user details and message data. MongoDB provides efficient data storage and retrieval, making the system scalable and flexible. The application can be executed on Windows operating system and accessed through web browsers such as **Google Chrome, Microsoft Edge, or Mozilla Firefox**.

These software tools help in developing and running the system smoothly while ensuring secure communication and proper data management.

2.2 Hardware Requirements

Hardware requirements define the physical components needed to run the Secure Messaging System using Hashing properly. The system does not require high-end hardware and can run efficiently on a basic computer system.

The system requires a computer with at least an **Intel Core i3 processor or equivalent**, which is sufficient for handling application processes and secure communication tasks. A **minimum of 4 GB RAM** is required to ensure smooth execution of the application without performance issues.

The system also requires a hard disk with at least **20 GB** of free space for storing software, database, and project files. Basic input devices such as a keyboard and mouse are required for user interaction, and a monitor is required to display the application interface.

These hardware components are sufficient for developing and running the system efficiently while ensuring smooth and secure operation.

2.3 Functional Requirements

Functional requirements define the functions and operations that the system must perform. These requirements describe what the system should do. The Secure Messaging System using Hashing provides different functionalities for users to ensure secure communication and data integrity.

The system allows users to register and login securely into the system. After successful login, users can access the dashboard, view available users, and select a user to start communication. The system allows users to send and receive messages in a secure manner. Each message is processed using hashing techniques before storage to maintain data integrity.

The system verifies the integrity of messages at the receiver's end. It compares the hash value and displays whether the message is secure or tampered. This helps users to trust the communication process. The system also provides features such as editing profile details, changing password, and managing basic settings.

The system performs important functions such as storing user and message data in the database, retrieving data when required, validating user credentials during login, and maintaining secure session management. These functions ensure that the system works efficiently and securely.

These functionalities help in providing secure communication, protecting data from unauthorized modification, and demonstrating the practical use of cybersecurity techniques.

2.4 Non-Functional Requirements

Non-functional requirements define the quality and performance aspects of the system. These requirements ensure that the system works properly, securely, and efficiently.

The system should provide good performance by responding quickly to user requests such as login, sending messages, and retrieving messages without delay. Efficient processing ensures smooth communication between users.

The system should provide strong security by allowing only authorized users to access the system. User credentials such as passwords should be protected using secure techniques, and messages should be processed using hashing to maintain data integrity and prevent unauthorized modification.

The system should be reliable, which means it should function correctly without failures and should store and retrieve data accurately. The system must ensure that messages and user data are not lost or corrupted.

The system should also be user-friendly, so that users can easily understand and use the system without any difficulty. A simple and clean interface helps users to perform actions smoothly.

The system should be maintainable, which means it should be easy to update, modify, and enhance in the future. This allows the addition of new security features or system improvements when required.

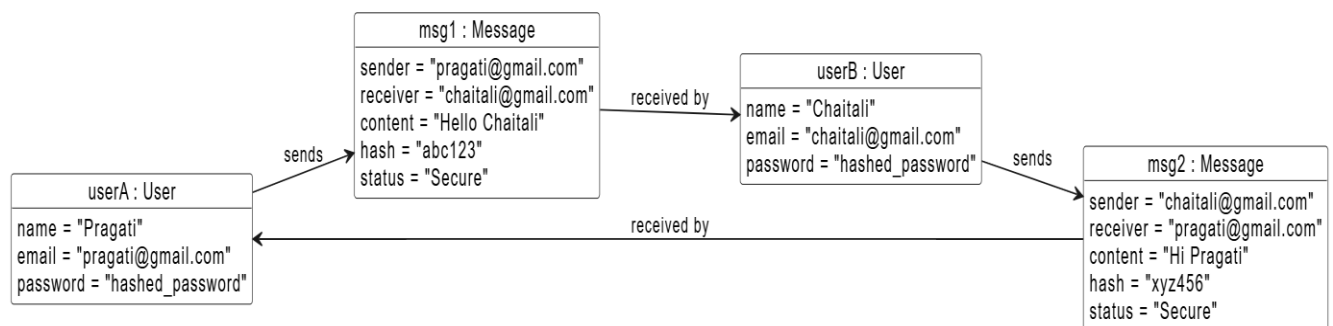
The system should also be available whenever required, so that users can access it anytime without interruption. High availability ensures continuous and reliable communication.

These non-functional requirements ensure that the system provides better performance, enhanced security, reliability, and overall usability.

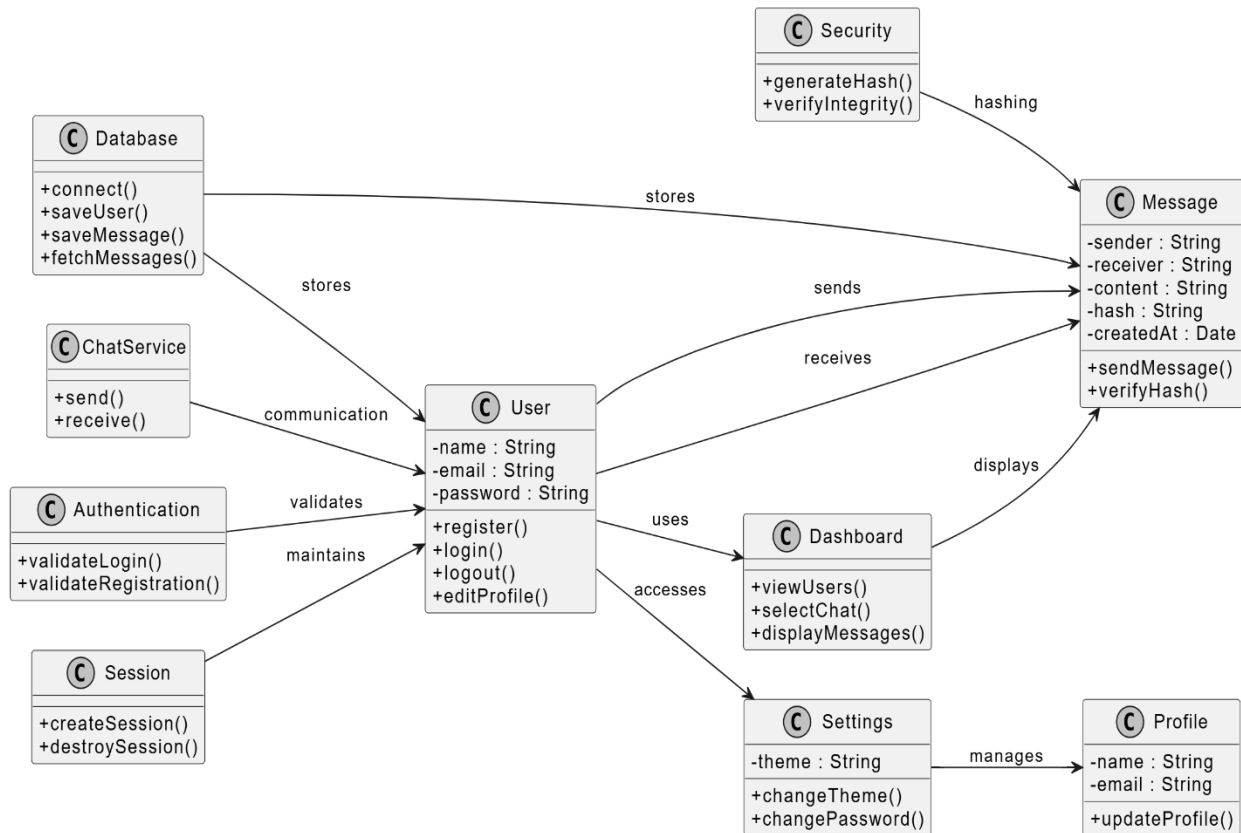
CHAPTER 3

3. Analysis Phase

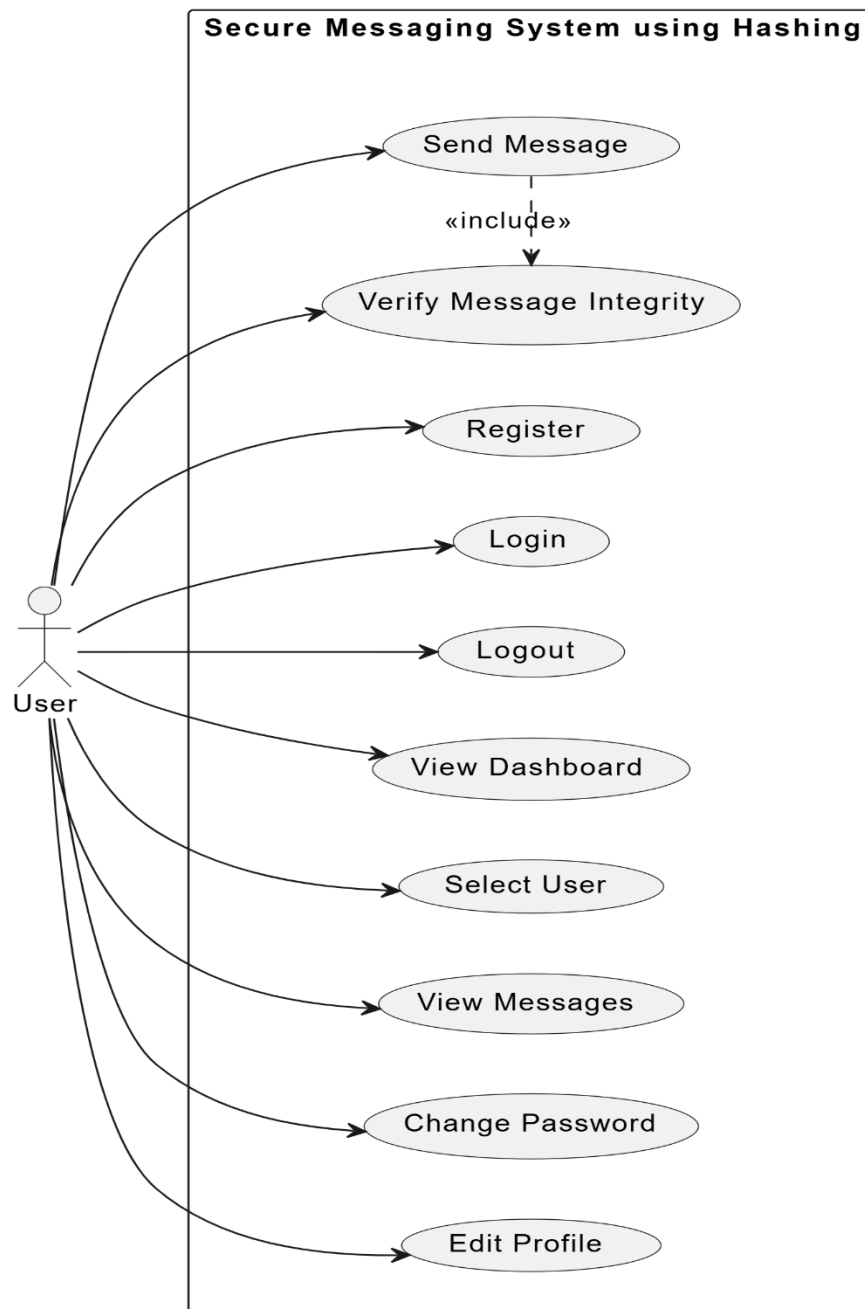
3.1 Object Diagram



3.2 Class Diagram



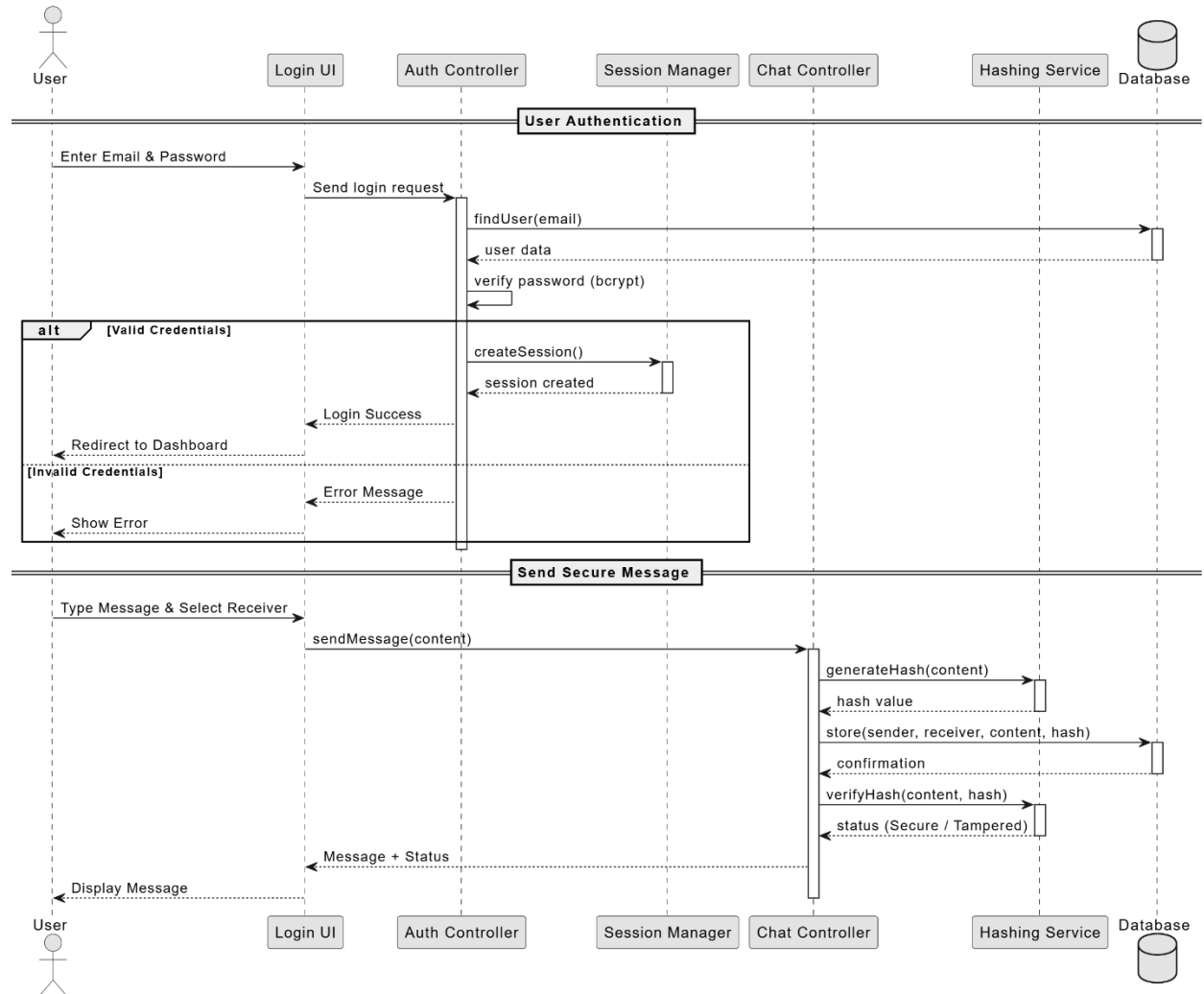
3.3 Use Case diagram



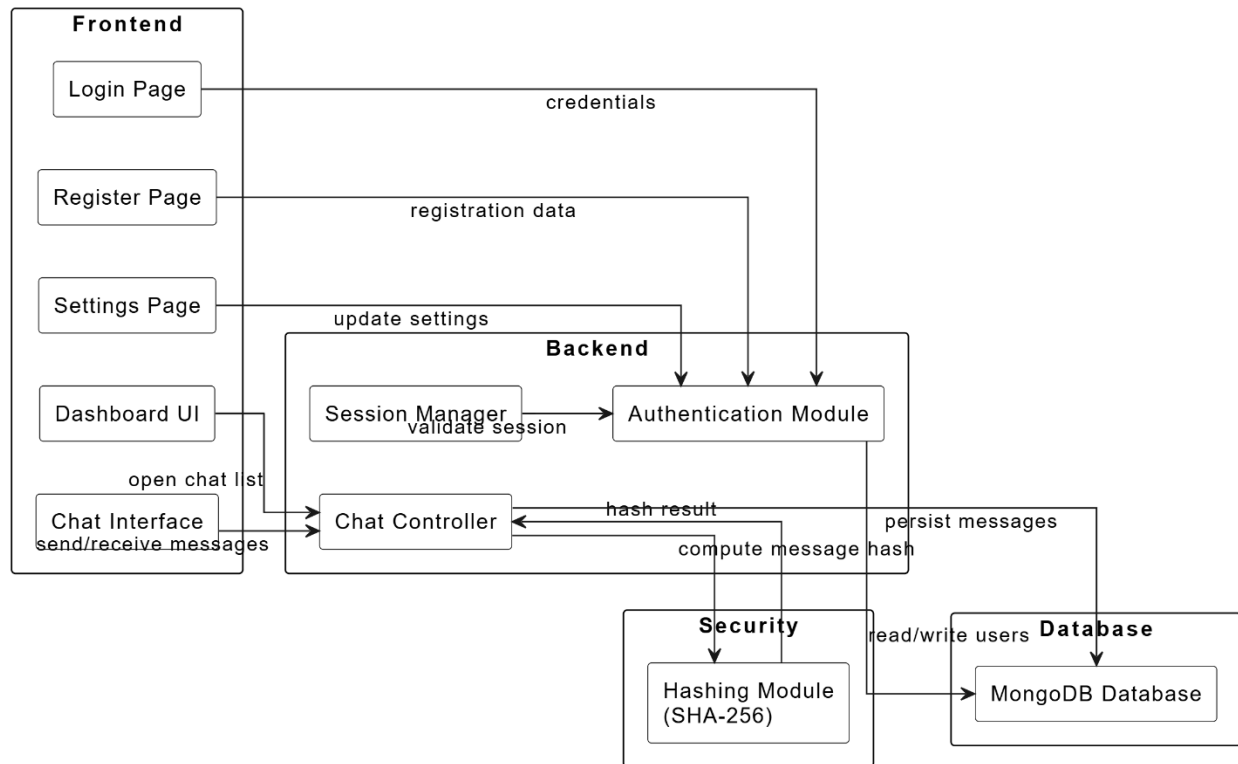
CHAPTER 4

4.Design Phase

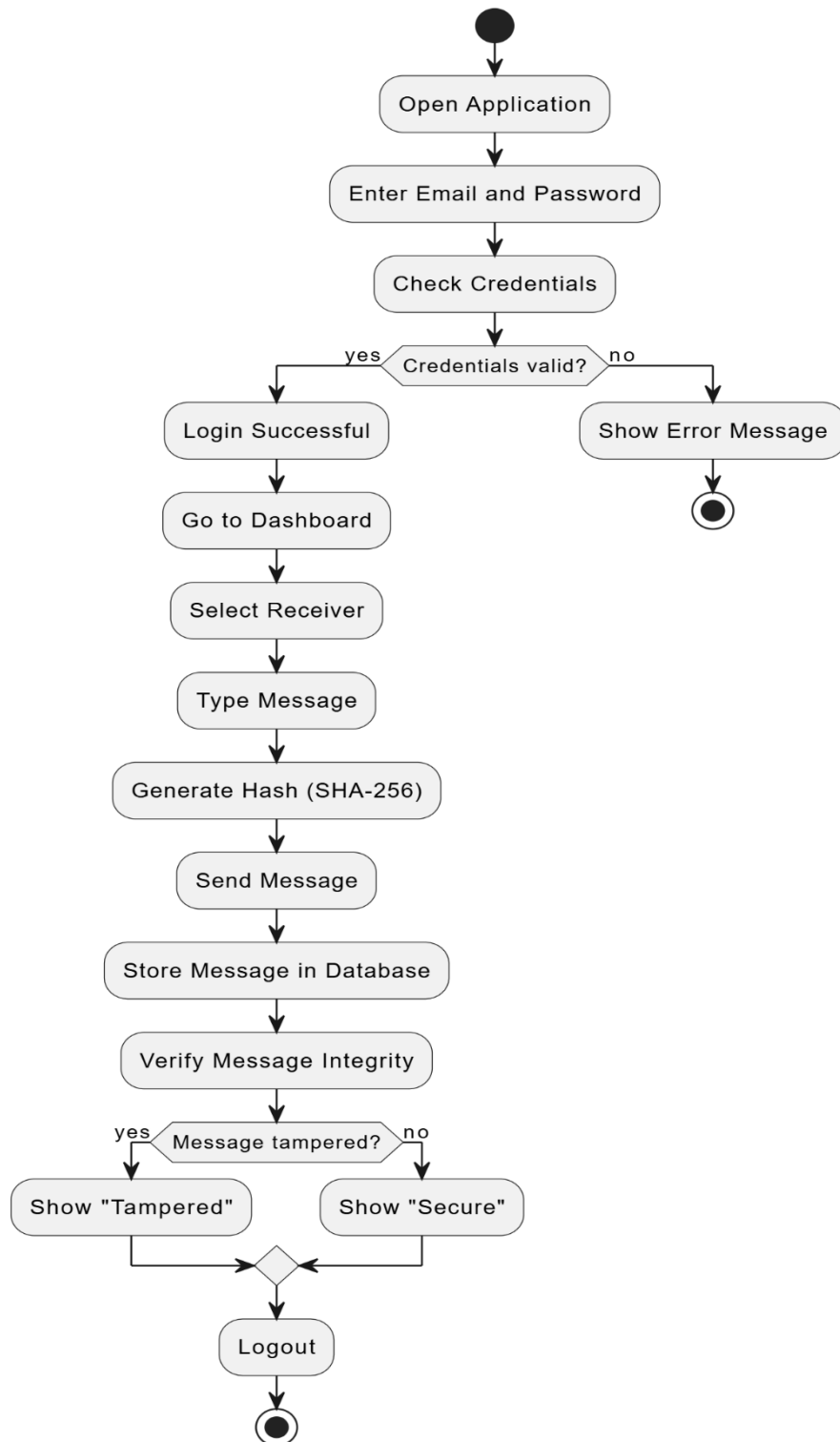
4.1 Sequence Diagram



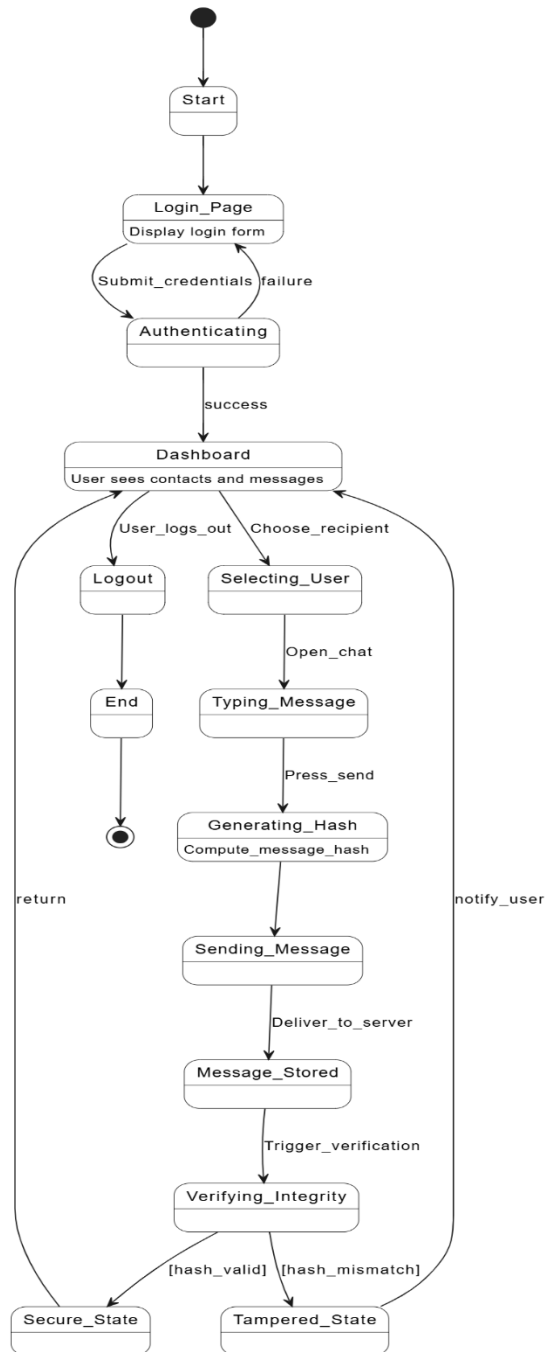
4.2 Component Diagram



4.3 Activity Diagram



4.4 State Chart Diagram



4.5 Web Site Map Diagram

